AI Lab Task 4

Name : Mehr Ali BSCS-4B

Greedy Best-First Search on Romania Map

In this task we implement the Greedy Best-First Search algorithm to find the shortest route from any city to Bucharest.

Greedy Best-First Search selects the node that appears to be closest to the goal using a heuristic function h(n).

Goal City: Bucharest

In [8]:
```
graph = {
    'Arad': {'Zerind': 75, 'Sibiu': 140, 'Timisoara': 118},
    'Zerind': {'Arad': 75, 'Oradea': 71},
    'Oradea': {'Zerind': 71, 'Sibiu': 151},
    'Sibiu': {'Arad': 140, 'Oradea': 151, 'Fagaras': 99, 'Rimnicu Vilcea': 80},
    'Timisoara': {'Arad': 118, 'Lugoj': 111},
    'Lugoj': {'Timisoara': 111, 'Mehadia': 70},
    'Mehadia': {'Lugoj': 70, 'Dobreta': 75},
    'Dobreta': {'Mehadia': 75, 'Craiova': 120},
    'Craiova': {'Dobreta': 120, 'Rimnicu Vilcea': 146, 'Pitesti': 138},
    'Rimnicu Vilcea': {'Sibiu': 80, 'Craiova': 146, 'Pitesti': 97},
    'Fagaras': {'Sibiu': 99, 'Bucharest': 211},
    'Pitesti': {'Rimnicu Vilcea': 97, 'Craiova': 138, 'Bucharest': 101},
    'Bucharest': {'Fagaras': 211, 'Pitesti': 101, 'Giurgiu': 90, 'Urziceni': 85},
    'Giurgiu': {'Bucharest': 90},
    'Urziceni': {'Bucharest': 85, 'Hirsova': 98, 'Vaslui': 142},
    'Hirsova': {'Urziceni': 98, 'Eforie': 86},
    'Eforie': {'Hirsova': 86},
    'Vaslui': {'Urziceni': 142, 'Iasi': 92},
    'Iasi': {'Vaslui': 92, 'Neamt': 87},
    'Neamt': {'Iasi': 87}
}

heuristic = {
    'Arad': 366,'Zerind': 374,'Oradea': 380,'Sibiu': 253,
    'Timisoara': 329,'Lugoj': 244,'Mehadia': 241,'Dobreta': 242,
    'Craiova': 160,'Rimnicu Vilcea': 193,'Fagaras': 176,
    'Pitesti': 100,'Bucharest': 0,'Giurgiu': 77,'Urziceni': 80,
    'Hirsova': 151,'Eforie': 161,'Vaslui': 199,'Iasi': 226,'Neamt': 234
}
```

In [9]:
```
from queue import PriorityQueue

def greedy_best_first_search(start, goal):
```

```
    visited = set()
    pq = PriorityQueue()

    pq.put((heuristic[start], start, [start]))

    while not pq.empty():

        h, current, path = pq.get()

        if current == goal:
            return path

        if current not in visited:

            visited.add(current)

            for neighbor in graph[current]:
                if neighbor not in visited:
                    pq.put((heuristic[neighbor], neighbor, path + [neighbor]))

    return None
```

In [10]:
```
start_city = "Arad"
goal_city = "Bucharest"

path = greedy_best_first_search(start_city, goal_city)

print("Path found:", path)
```

```
Path found: ['Arad', 'Sibiu', 'Fagaras', 'Bucharest']
```

In [11]:
```
def calculate_distance(path):

    total = 0

    for i in range(len(path)-1):
        total += graph[path[i]][path[i+1]]

    return total

print("Total Distance:", calculate_distance(path))
```

```
Total Distance: 450
```